Exercício 9.4: Desafio: modelo RBAC completo para quatro perfis em dois namespaces

Aluno: Luiz Brito · **Data:** 24/10/2026 · **Disciplina:** Cloud Native Development (CP3)

Ambiente

- Cluster kind com 3 nós, Kubernetes v1.36.1, namespaces `store-dev` e `store-prod`.
- Um usuário por certificado para cada perfil, emitido pelo mesmo fluxo do 9.2 (CSR `kubernetes.io/kube-apiserver-client`, 7 dias). As chaves ficam fora do repositório.

```
ana.dev:      subject=CN=ana.dev,O=fiap:dev
bia.qa:       subject=CN=bia.qa,O=fiap:qa
carlos.sre:   subject=CN=carlos.sre,O=fiap:sre
duda.auditor: subject=CN=duda.auditor,O=fiap:auditor
```

Desenho

Perfil	`store-dev`	`store-prod`	Cluster
`fiap:dev`	`store-workloads-escrita`	`view`	nada
`fiap:qa`	`view` + `store-qa-testes`	`view`	nada
`fiap:sre`	`admin`	`admin`	`store-sre-cluster-leitura` (único ClusterRoleBinding)
`fiap:auditor`	`store-auditor`	`store-auditor`	nada

Quatro ClusterRoles próprias e três built-in (`view`, `admin`), ligadas por **11 RoleBindings e 1 ClusterRoleBinding**. Toda regra fica numa ClusterRole e o escopo vem do binding. Por isso dev e prod reaproveitam as mesmas regras sem cópia. As decisões estão comentadas no YAML:

- **dev em prod =** `view`**:** o built-in já exclui Secrets e `pods/exec`, que é exatamente o pedido.
- **SRE =** `admin` **por namespace, não** `cluster-admin`**:** `admin` cobre Secrets, PVCs, StatefulSets, DaemonSets e o RBAC **do namespace**, sem nenhum poder sobre o cluster. O pouco que o SRE precisa ver fora dos namespaces (nodes, PVs, StorageClasses, Namespaces) está numa ClusterRole só de leitura. Esse é o único caso legítimo de `ClusterRoleBinding`, porque esses recursos não têm namespace.
- **dev sem Secrets mesmo em** `store-dev`**:** o enunciado lista os workloads do dev e Secrets não está entre eles. Credenciais ficam com o SRE.

```yaml
# Modelo RBAC da FIAP Store: 4 perfis (grupos) x 2 namespaces.
# Regras gerais:
# - regras ficam em ClusterRoles (sem escopo) e o escopo vem dos RoleBindings,
#   um por namespace autorizado; assim a mesma regra serve dev e prod sem copia
# - nenhum curinga em verbs ou resources: tudo explicito
# - ClusterRoleBinding so para o que e cluster-wide de verdade (leitura de nos etc. do SRE)

# ------------------------------------------------------------------------
# ClusterRoles
# ------------------------------------------------------------------------

# Workloads de aplicacao com escrita. Sem Secrets, PVCs, StatefulSets e DaemonSets
# (sao do SRE) e sem RBAC (ninguem do time de app se autopromove).
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: store-workloads-escrita
rules:
  - apiGroups: [""]
    resources: ["pods", "services", "configmaps"]
    verbs: ["get", "list", "watch", "create", "update", "patch", "delete"]
  - apiGroups: [""]
    resources: ["pods/log"]
    verbs: ["get"]
  - apiGroups: [""]
    resources: ["pods/exec"]
    verbs: ["create"]
  - apiGroups: [""]
    resources: ["endpoints", "events"]
    verbs: ["get", "list", "watch"]
  - apiGroups: ["apps"]
    resources: ["deployments", "deployments/scale"]
    verbs: ["get", "list", "watch", "create", "update", "patch", "delete"]
  - apiGroups: ["apps"]
    resources: ["replicasets"]
    verbs: ["get", "list", "watch"]
  - apiGroups: ["batch"]
    resources: ["jobs", "cronjobs"]
    verbs: ["get", "list", "watch", "create", "update", "patch", "delete"]
  - apiGroups: ["autoscaling"]
    resources: ["horizontalpodautoscalers"]
    verbs: ["get", "list", "watch", "create", "update", "patch", "delete"]
---
# Extras do QA, so para store-dev: rodar suites como Jobs e entrar nos Pods.
# A leitura vem do ClusterRole "view" (built-in), ligado a parte.
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: store-qa-testes
rules:
  - apiGroups: ["batch"]
    resources: ["jobs"]
    verbs: ["create", "delete"]
  - apiGroups: [""]
    resources: ["pods/exec"]
    verbs: ["create"]
---
# Leitura cluster-wide do SRE. Recursos sem namespace: so um ClusterRoleBinding resolve.
```

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: store-sre-cluster-leitura
rules:
  - apiGroups: [""]
    resources: ["nodes", "persistentvolumes", "namespaces"]
    verbs: ["get", "list", "watch"]
  - apiGroups: ["storage.k8s.io"]
    resources: ["storageclasses"]
    verbs: ["get", "list", "watch"]
---
# Auditor: leitura de tudo que importa nos namespaces, inclusive RBAC, e nunca Secrets.
# Nao usa "view" porque view nao inclui roles/rolebindings (ver MATRIZ.md).
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: store-auditor
rules:
  - apiGroups: [""]
    resources: ["pods", "pods/log", "services", "endpoints", "configmaps", "events",
                "persistentvolumeclaims", "serviceaccounts", "resourcequotas", "limitranges"]
    verbs: ["get", "list", "watch"]
  - apiGroups: ["apps"]
    resources: ["deployments", "replicasets", "statefulsets", "daemonsets"]
    verbs: ["get", "list", "watch"]
  - apiGroups: ["batch"]
    resources: ["jobs", "cronjobs"]
    verbs: ["get", "list", "watch"]
  - apiGroups: ["autoscaling"]
    resources: ["horizontalpodautoscalers"]
    verbs: ["get", "list", "watch"]
  - apiGroups: ["policy"]
    resources: ["poddisruptionbudgets"]
    verbs: ["get", "list", "watch"]
  - apiGroups: ["networking.k8s.io"]
    resources: ["ingresses", "networkpolicies"]
    verbs: ["get", "list", "watch"]
  - apiGroups: ["rbac.authorization.k8s.io"]
    resources: ["roles", "rolebindings"]
    verbs: ["get", "list", "watch"]

# ------------------------------------------------------------------------------
# fiap:dev → escrita em store-dev, view (built-in) em store-prod.
# view ja exclui Secrets e pods/exec, que e exatamente o pedido para prod.
# ------------------------------------------------------------------------------
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: dev-workloads
  namespace: store-dev
subjects:
  - kind: Group
    name: fiap:dev
    apiGroup: rbac.authorization.k8s.io
roleRef:
  apiGroup: rbac.authorization.k8s.io
```

```yaml
  kind: ClusterRole
  name: store-workloads-escrita
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: dev-view
  namespace: store-prod
subjects:
  - kind: Group
    name: fiap:dev
    apiGroup: rbac.authorization.k8s.io
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: view

# ------------------------------------------------------------------------
# fiap:qa → view nos dois namespaces + Jobs e exec so em store-dev
# ------------------------------------------------------------------------
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: qa-view
  namespace: store-dev
subjects:
  - kind: Group
    name: fiap:qa
    apiGroup: rbac.authorization.k8s.io
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: view
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: qa-view
  namespace: store-prod
subjects:
  - kind: Group
    name: fiap:qa
    apiGroup: rbac.authorization.k8s.io
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: view
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: qa-testes
  namespace: store-dev
subjects:
  - kind: Group
    name: fiap:qa
    apiGroup: rbac.authorization.k8s.io
```

```yaml
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: store-qa-testes

# ------------------------------------------------------------------------
# fiap:sre -> admin (built-in) por namespace + leitura cluster-wide.
# admin cobre Secrets, PVCs, StatefulSets, DaemonSets e RBAC do namespace, sem
# nenhum poder sobre o cluster (nao cria namespaces, nodes, ClusterRoles).
# cluster-admin seria um ClusterRoleBinding com curinga em tudo: descartado.
# ------------------------------------------------------------------------
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: sre-admin
  namespace: store-dev
subjects:
  - kind: Group
    name: fiap:sre
    apiGroup: rbac.authorization.k8s.io
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: admin
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: sre-admin
  namespace: store-prod
subjects:
  - kind: Group
    name: fiap:sre
    apiGroup: rbac.authorization.k8s.io
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: admin
---
# unico ClusterRoleBinding do modelo: nodes, PVs, StorageClasses e Namespaces nao tem namespace
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: sre-cluster-leitura
subjects:
  - kind: Group
    name: fiap:sre
    apiGroup: rbac.authorization.k8s.io
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: store-sre-cluster-leitura

# ------------------------------------------------------------------------
# fiap:auditor -> store-auditor nos dois namespaces, nada mais
# ------------------------------------------------------------------------
---
```

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: auditor
  namespace: store-dev
subjects:
  - kind: Group
    name: fiap:auditor
    apiGroup: rbac.authorization.k8s.io
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: store-auditor
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: auditor
  namespace: store-prod
subjects:
  - kind: Group
    name: fiap:auditor
    apiGroup: rbac.authorization.k8s.io
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: store-auditor
```

Nenhum curinga:

```
$ grep -c '\*' 01-rbac-modelo.yaml
0
```

Script

```bash
#!/usr/bin/env bash
# Gera a matriz de permissoes a partir de "kubectl auth can-i" reais.
# Uso: ./02-testa-rbac.sh > 03-matriz-gerada.md
set -euo pipefail

perfis="dev qa sre auditor"
namespaces="store-dev store-prod"
# recursos sem namespace: testados sem -n (com -n o can-i responde sobre
# o proprio objeto Namespace e da "yes" falso para list namespaces via view)
cluster_scoped="nodes namespaces storageclasses.storage.k8s.io clusterrolebindings.rbac.authorization.k8s.io"
# "verbo recurso [subresource]"
checks=(
  "get pods"
  "create pods"
  "get pods log"
  "create pods exec"
  "patch services"
  "get configmaps"
  "create deployments.apps"
```

```
  "delete deployments.apps"
  "create jobs.batch"
  "delete jobs.batch"
  "create horizontalpodautoscalers.autoscaling"
  "get secrets"
  "list secrets"
  "create secrets"
  "create persistentvolumeclaims"
  "create statefulsets.apps"
  "create daemonsets.apps"
  "list roles.rbac.authorization.k8s.io"
  "list rolebindings.rbac.authorization.k8s.io"
  "create rolebindings.rbac.authorization.k8s.io"
  "get nodes"
  "list namespaces"
  "list storageclasses.storage.k8s.io"
  "create clusterrolebindings.rbac.authorization.k8s.io"
)

printf '| verbo recurso |'
for g in $perfis; do for ns in $namespaces; do printf ' %s<br>%s |' "$g" "${ns#store-}"; done; done
printf '\n|---|'
for g in $perfis; do for ns in $namespaces; do printf '%s' '---|'; done; done
printf '\n'

for c in "${checks[@]}"; do
  read -r verbo recurso sub <<<"$c"
  printf '| `%s %s%s` |' "$verbo" "$recurso" "${sub:+/$sub}"
  for g in $perfis; do
    for ns in $namespaces; do
      escopo=(-n "$ns")
      [[ " $cluster_scoped " == *" $recurso "* ]] && escopo=()
      r=$(kubectl auth can-i "$verbo" "$recurso" ${sub:+--subresource=$sub} "${escopo[@]}" \
            --as="usuario.$g" --as-group="fiap:$g" 2>/dev/null || true)
      printf ' %s |' "$r"
    done
  done
  printf '\n'
done
```

Achado ao escrever o script: na primeira versão, todos os checks passavam `-n <namespace>`, e `list namespaces` deu `yes` para dev (prod) e qa (dev e prod). É um falso positivo. Com `-n`, o `can-i` pergunta sobre o objeto Namespace **dentro** daquele namespace, e o `view` tem uma regra `namespaces: get/list/watch` que, via RoleBinding, só permite ler o próprio objeto Namespace. A requisição real mostra a diferença:

```
$ kubectl get namespaces --as=usuario.qa --as-group=fiap:qa
Error from server (Forbidden): namespaces is forbidden: User "usuario.qa" cannot list resource "namespaces" in API group
"" at the cluster scope
$ kubectl get namespace store-prod --as=usuario.qa --as-group=fiap:qa
NAME         STATUS   AGE
store-prod   Active   4m10s
```

Na versão final, recursos sem namespace são testados sem `-n`.

MATRIZ

Saída de `./02-testa-rbac.sh`, sem edição (24 linhas × 8 colunas):

verbo recurso	dev dev	dev prod	qa dev	qa prod	sre dev	sre prod	auditor dev	auditor prod
`get pods`	yes	yes	yes	yes	yes	yes	yes	yes
`create pods`	yes	no	no	no	yes	yes	no	no
`get pods/log`	yes	yes	yes	yes	yes	yes	yes	yes
`create pods/exec`	yes	no	yes	no	yes	yes	no	no
`patch services`	yes	no	no	no	yes	yes	no	no
`get configmaps`	yes	yes	yes	yes	yes	yes	yes	yes
`create deployments.apps`	yes	no	no	no	yes	yes	no	no
`delete deployments.apps`	yes	no	no	no	yes	yes	no	no
`create jobs.batch`	yes	no	yes	no	yes	yes	no	no
`delete jobs.batch`	yes	no	yes	no	yes	yes	no	no
`create horizontalpodautoscalers.autoscaling`	yes	no	no	no	yes	yes	no	no
`get secrets`	no	no	no	no	yes	yes	no	no
`list secrets`	no	no	no	no	yes	yes	no	no
`create secrets`	no	no	no	no	yes	yes	no	no
`create persistentvolumeclaims`	no	no	no	no	yes	yes	no	no
`create statefulsets.apps`	no	no	no	no	yes	yes	no	no
`create daemonsets.apps`	no	no	no	no	yes	yes	no	no
`list roles.rbac.authorization.k8s.io`	no	no	no	no	yes	yes	yes	yes
`list rolebindings.rbac.authorization.k8s.io`	no	no	no	no	yes	yes	yes	yes
`create rolebindings.rbac.authorization.k8s.io`	no	no	no	no	yes	yes	no	no
`get nodes`	no	no	no	no	yes	yes	no	no
`list namespaces`	no	no	no	no	yes	yes	no	no
`list storageclasses.storage.k8s.io`	no	no	no	no	yes	yes	no	no
`create clusterrolebindings.rbac.authorization.k8s.io`	no	no	no	no	no	no	no	no

Checagens dos critérios na matriz:

- `auditor`: `no` em todos os verbos de escrita (`create`, `patch`, `delete`) e em `get`/`list secrets`, nos dois namespaces. `yes` em `list roles` e `list rolebindings`.
- `dev`: `create pods/exec` é `yes` em `store-dev` e `no` em `store-prod`.
- `sre`: `get nodes` é `yes` e `create clusterrolebindings` é `no`.

- `qa`: `create` / `delete jobs` e `create pods/exec` só em `store-dev`.

Evidência com os certificados

O `can-i` usa impersonation. Para fechar, uma requisição real de cada perfil com o próprio certificado. As escritas usam `--dry-run=server`, que passa pela autorização mas não persiste nada:

```
$ [ana.dev] kubectl create deployment teste --image=store-api:1.0 -n store-dev --dry-run=server
deployment.apps/teste created (server dry run)
$ [ana.dev] kubectl get pods -n store-prod
NAME   READY   STATUS    RESTARTS   AGE
alvo   1/1     Running   0          4m59s
$ [ana.dev] kubectl get secrets -n store-prod
Error from server (Forbidden): secrets is forbidden: User "ana.dev" cannot list resource "secrets" in API group "" in the
namespace "store-prod"
$ [ana.dev] kubectl exec -n store-prod alvo -- id
error: unable to upgrade connection: pods "alvo" is forbidden: User "ana.dev" cannot create resource "pods/exec" in API
group "" in the namespace "store-prod"

$ [bia.qa] kubectl create job suite --image=busybox:1.37 -n store-dev --dry-run=server
job.batch/suite created (server dry run)
$ [bia.qa] kubectl create job suite --image=busybox:1.37 -n store-prod --dry-run=server
error: failed to create job: jobs.batch is forbidden: User "bia.qa" cannot create resource "jobs" in API group "batch" in
the namespace "store-prod"

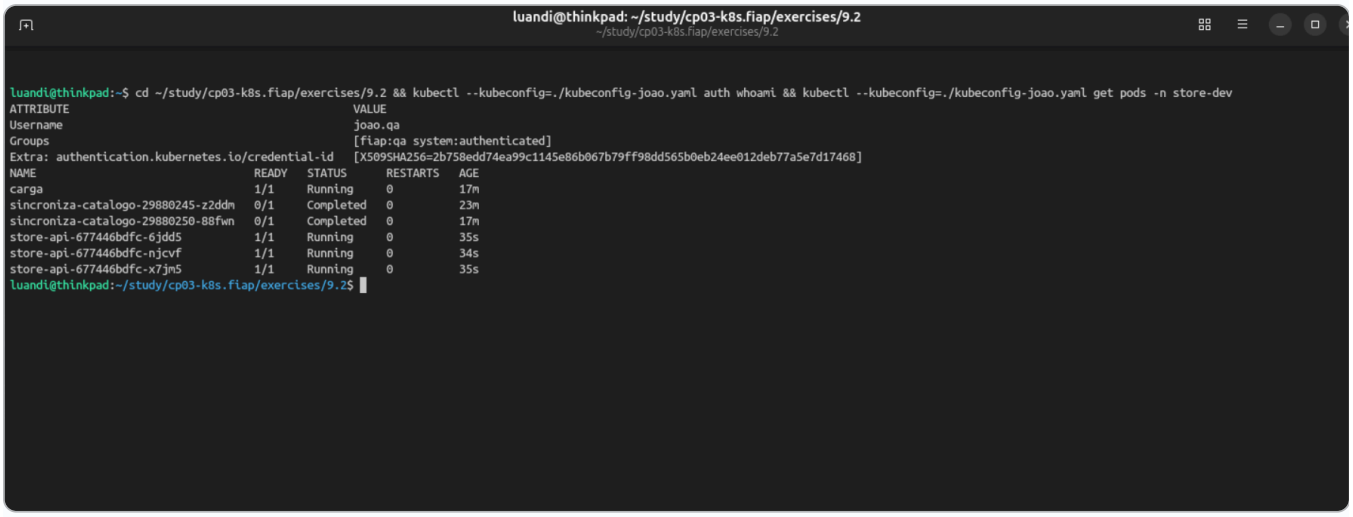
$ [carlos.sre] kubectl get nodes
NAME                       STATUS   ROLES           AGE     VERSION
fiap-store-control-plane   Ready    control-plane   4d14h   v1.36.1
fiap-store-worker          Ready    <none>          4d14h   v1.36.1
fiap-store-worker2         Ready    <none>          4d14h   v1.36.1
$ [carlos.sre] kubectl get secrets -n store-prod
No resources found in store-prod namespace.
$ [carlos.sre] kubectl create clusterrolebinding eu-admin --clusterrole=cluster-admin --user=carlos.sre --dry-run=server
error: failed to create clusterrolebinding: clusterrolebindings.rbac.authorization.k8s.io is forbidden: User "carlos.sre"
cannot create resource "clusterrolebindings" in API group "rbac.authorization.k8s.io" at the cluster scope

$ [duda.auditor] kubectl get rolebindings -n store-prod
NAME        ROLE                        AGE
auditor     ClusterRole/store-auditor   89s
dev-view    ClusterRole/view            89s
qa-view     ClusterRole/view            89s
$ [duda.auditor] kubectl get secrets -n store-dev
Error from server (Forbidden): secrets is forbidden: User "duda.auditor" cannot list resource "secrets" in API group ""
in the namespace "store-dev"
$ [duda.auditor] kubectl delete pod alvo -n store-prod --dry-run=server
Error from server (Forbidden): pods "alvo" is forbidden: User "duda.auditor" cannot delete resource "pods" in API group
"" in the namespace "store-prod"
```

O SRE tenta se promover a `cluster-admin` e recebe `Forbidden`. Isso confirma que "controle total nos namespaces" não virou controle do cluster.

Efeito colateral real, e desejado, do modelo por grupo: o `joao.qa` do exercício 9.2 tinha perdido todo o acesso na revogação. Depois que este modelo foi aplicado, ele voltou a listar Pods em `store-dev` sem que nenhum binding citasse o nome dele, só porque o certificado tem `O=fiap:qa`. É o comportamento

esperado de RBAC por grupo, e também o motivo de a emissão de certificados precisar de controle: quem assina um CSR com `O=fiap:sre` está concedendo o perfil SRE.



Por que o auditor não usa o ClusterRole `view`

O `view` foi desenhado para "ver a aplicação", não para auditar, e falha nos dois sentidos do requisito. **Falta** o que o auditor precisa: `view` não inclui `roles` nem `rolebindings`, e auditar quem tem acesso a quê é o centro do trabalho de auditoria. Ele também **muda sozinho**: `view` é um ClusterRole agregado (`aggregate-to-view`), e qualquer CRD instalado depois (KEDA, Argo CD) pode adicionar recursos a ele sem ninguém revisar a regra do auditor. Por isso foi criada a `store-auditor`, com a lista fechada e explícita de recursos, incluindo RBAC e excluindo `secrets`. A leitura de `rolebindings` é privilégio sensível, porque entrega o mapa de quem pode fazer o quê (útil para quem planeja escalar privilégio). Ela foi concedida porque é a função do perfil, mas **só leitura**, só nos dois namespaces e sem `clusterrolebindings`. Não foi usado `resourceNames` porque o auditor precisa ver todos os bindings do namespace, inclusive os criados depois, e uma lista de nomes fixos deixaria de fora justamente o binding novo que mais interessa auditar.

Critérios de aceite

Critério	Resultado
Matriz com no mínimo 14 linhas × 8 colunas, de saídas reais	ok (24 × 8)
`auditor`: `no` em toda escrita e em `get secrets` nos dois namespaces	ok
`dev`: `create pods/exec` `yes` em `store-dev` e `no` em `store-prod`	ok
`sre`: `get nodes` `yes`, `create clusterrolebindings` `no`	ok
Nenhum `"*"` em `verbs` ou `resources`	ok (`grep` = 0)